Human-Centered Neuroengineering LAB

C Programming (1)

1st week. Lab01 hello, world!

Department of Embedded System Engineering
Incheon National University

Instructor: Kyungho Won
kyungho.won@inu.ac.kr
Human-Centered Neuroengineering (HCN) Lab

INU 인천대학교 INCHEON NATIONAL UNIVERSITY

* These lecture slides are for class use only. Unauthorized distribution or modification is prohibited.

General Course Policy

- Class hours (306)
  - Tue. 11h-13h / Thr. 16h-18h

- TA
  - 황주옥 (ghkdwndhr227@inu.ac.kr), 010-6403-1470
  - 연락 시 00 과목을 수강중인 000 입니다 밝힌 후 질의

- Office hours (instructor's office: Building #9, Room 422)
  - Appointment is necessary in advance
  - E-mail: kyungho.won@inu.ac.kr
  - 연락 시 00 과목을 수강중인 000 입니다 밝힌 후 질의

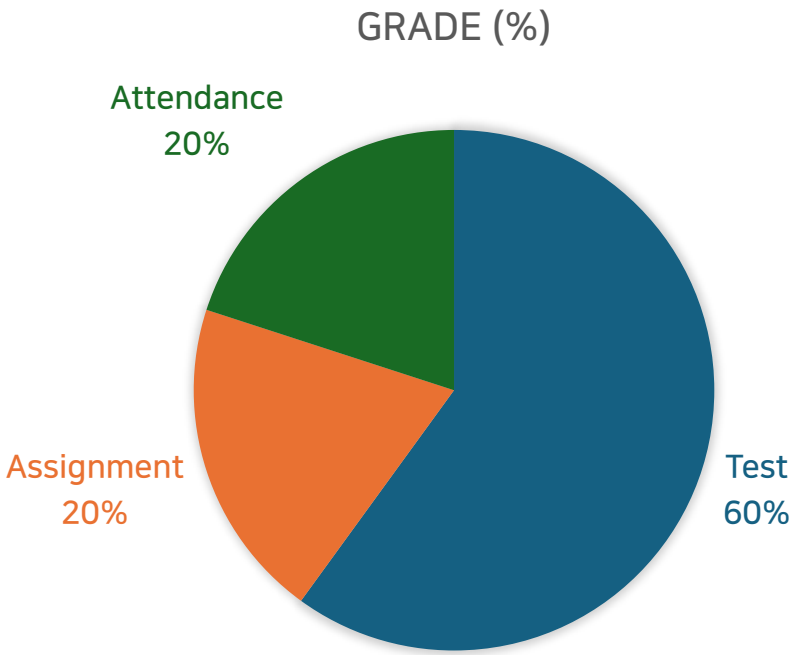
General Course Policy

- Grade (상대평가)
  - Exam A/Exam B/Exam C (Test or Projects): 60 %
  - Assignments: 20 %
  - Attendance: 20 %
    - 1시간 결석 시 1/3점 감점, 3시간 결석 시 1점 감점
    - 실제 수업시간 수의 1/3이상 결석: 학점인정 불가

- 상대평가 등급 비율 (정정기간 후 최종 수강인원에 따라 일부 조정될 수 있음)

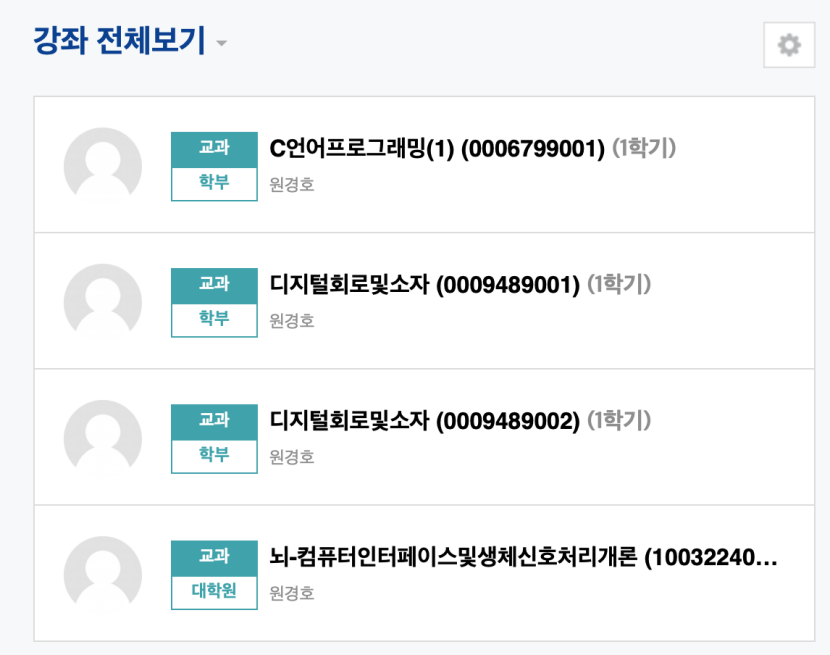
A+/A0등급	B+/B0등급	C+ 이하	F
상위 35 %	상위 35 % ~ 75%	75 % - 100 %	출결/과제 성실성

** A+/A0, B+/B0는 절반씩 배분



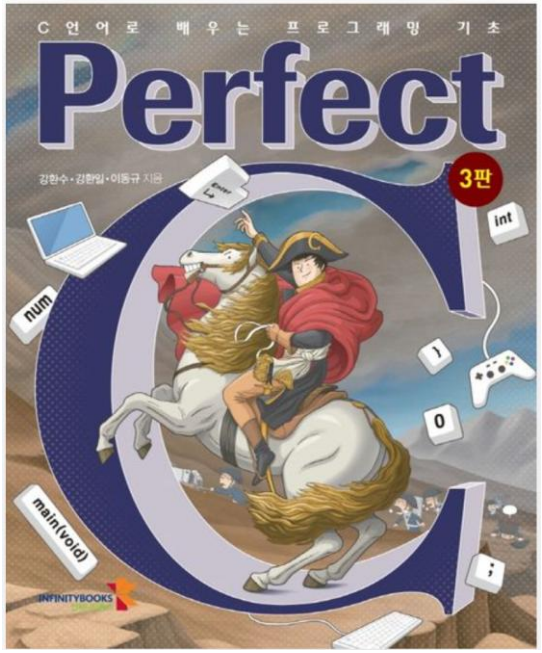
General Course Policy

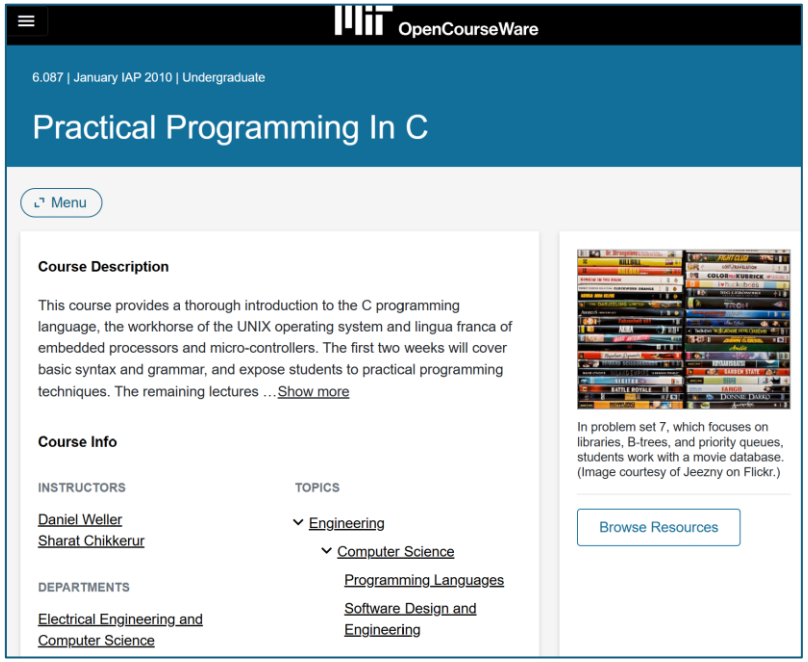
- e-learning site; 인천대 포탈 로그인 > 이러닝 선택 > 수강중인 강좌 선택
- App: 인천대학교 스마트캠퍼스, 코스모스 앱 다운로드 및 로그인
- 강의자료 및 과제, 기타 공지사항 확인



Class Materials

- Lecture slides
  - Slides (PDF) will be distributed 1-hour earlier
- Perfect C 3판
  - 강환수, 강환일 이동규 저
  - 인피니티북스
- Practical Programming in C
  - Daniel Weller, Sharat Chikkerur
  - MIT OpenCouseWare

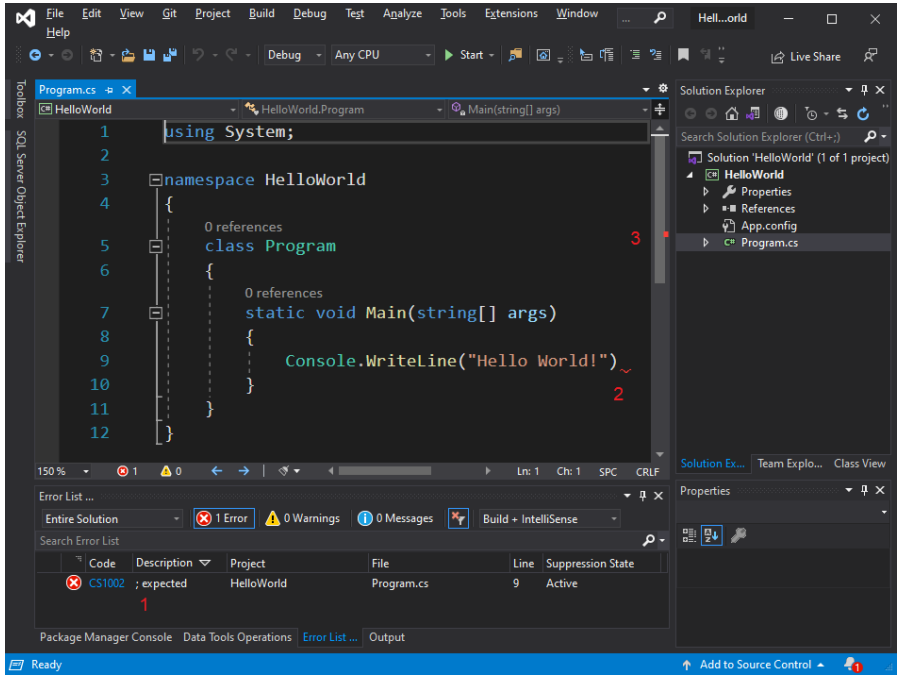




Course Syllabus (tentative)

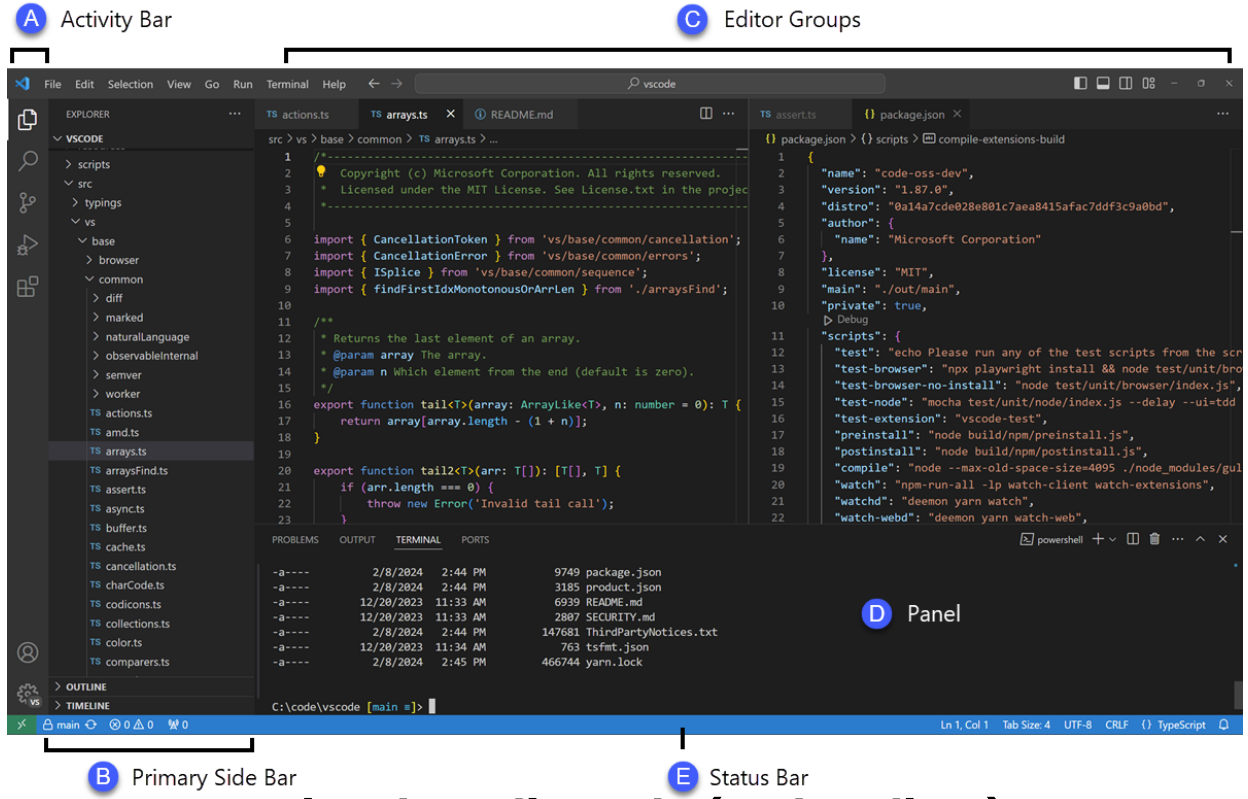
1st week 09.01 ~ 09.05	- Introduction, Hello World, Compiling & Debugging	
2nd week 09.08 ~ 09.12	- Variables and Data Types	
3rd week 09.15 ~ 09.19	- Operators, Expressions, Preprocessor & I/O	
4th week 09.22 ~ 09.26	- Control Flow – Conditions (if, switch)	
5th week 09.29 ~ 10.03	- Control Flow – Loops (for, while, do-while)	
6th week 10.06 ~ 10.10	- Arrays	
7th week 10.13 ~ 10.17	- Functions	
8th week 10.20 ~ 10.24	Midterm	

IDE & Code editor (used in this course) (Independent development environment)



Visual studio (IDE)

full IDE with built-in compiler, debugger, and build system. Click and run.



Visual studio code (code editor)

lightweight editor. Need to install compiler (gcc/clang) and configure everything yourself.

Windows: Visual Studio

- During installation, select:
- 'Desktop development with C++'
- (includes C compiler + debugger)

- Recommended for most Windows users
- Large but full-featured professional IDE

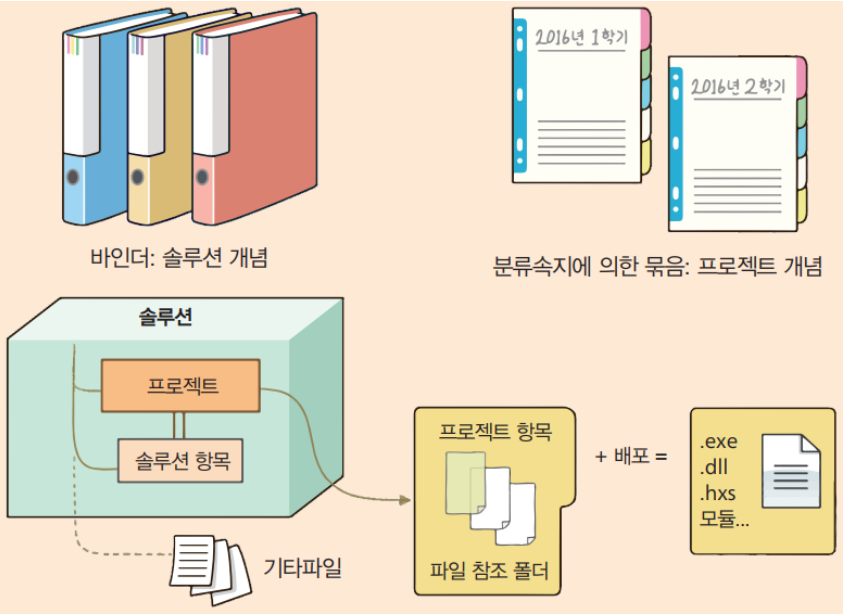
Visual Studio: First Project

1. File → New → Project
2. Select: 'Empty Project' (C++)
3. Project name: HelloWorld
4. Right-click Source Files → Add → New Item
5. Choose: C++ File (.cpp) name it: hello.c
6. Write your code and press Ctrl+F5 to run

- Tip: Save your file with .c extension!

Solution & Project in Visual Studio

- Visual Studio organizes code using Solutions and Projects
- Solution = a binder (container)
  - Holds one or more projects, like a folder for a course
- Project = a tab divider inside the binder
  - Contains source files for one program
  - Each project produces one executable (.exe)
- Example: Solution 'ch01' contains
  - Project 'P01-HelloWorld' → hello.c → P01-HelloWorld.exe
  - Project 'P02-Practice' → practice.c → P02-Practice.exe



Mac: Visual Studio Code

- Required extensions (install from Extensions tab):
- C/C++ by Microsoft

  → Syntax highlighting, IntelliSense, debugging
- Code Runner by Jun Han

  → Run C files with one click (▶ button)

VS Code: Build and Run

1. Open VS Code
2. File → Open Folder → choose your project folder
3. Create a new file: hello.c
4. Write your C code
5. Press the ▶ (Run) button or Ctrl+Alt+N

- Output appears in the terminal panel at the bottom
- Tip: Make sure to save (Cmd+S) before running!

The Write → Build → Run Cycle

- Write: Type C code in the editor

  ↓

- Build: IDE compiles and links (Ctrl+F5 / ▶)

  ↓

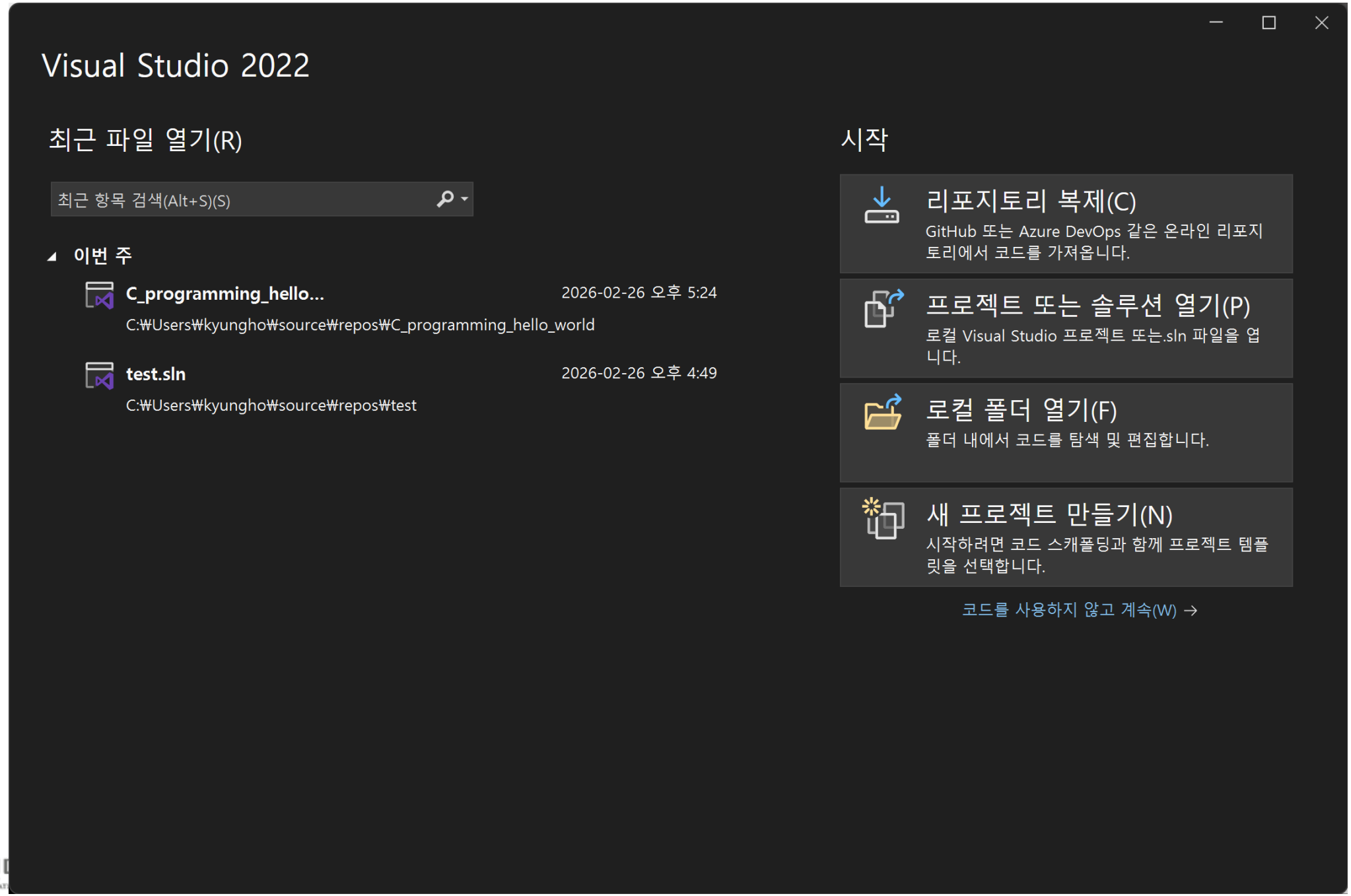
- Run: See the output

  ↓

- Error?: Read the error message, fix, and rebuild

- You will do this thousands of times — get comfortable!

Your First C Program


Visual Studio 2022
최근 파일 열기(R)
최근 항목 검색(Alt+S)(S)
이번 주
C_programming_hello...
2026-02-26 오후 5:24
C:₩Users₩kyungho₩source₩repos₩C_programming_hello_world
test.sln
2026-02-26 오후 4:49
C:₩Users₩kyungho₩source₩repos₩test
시작
리포지토리 복제(C)
GitHub 또는 Azure DevOps 같은 온라인 리포지토리에서 코드를 가져옵니다.
프로젝트 또는 솔루션 열기(P)
로컬 Visual Studio 프로젝트 또는.sln 파일을 엽니다.
로컬 폴더 열기(F)
폴더 내에서 코드를 탐색 및 편집합니다.
새 프로젝트 만들기(N)
시작하려면 코드 스캐폴딩과 함께 프로젝트 템플릿을 선택합니다.
코드를 사용하지 않고 계속(W) →

새 프로젝트 만들기

템플릿 검색(Alt+S)(S)

최근 프로젝트 템플릿(R)

빈 프로젝트 C++

빈 솔루션

모든 언어(L) 모든 플랫폼(P) 모든 프로젝트 형식(T)

Vue and ASP.NET Core 새로 만들기

프런트 엔드 Vue 프로젝트와 백 엔드 ASP.NET Core 프로젝트가 있는 전체 스택 애플리케이션

TypeScript Windows macOS Linux 웹

내보내기가 있는 동적 연결 라이브러리(DLL) 새로 만들기

실행 중인 여러 Windows 앱 간에 공유할 수 있는 .dll을 빌드하세요.

빈 프로젝트

Windows용 C++를 사용하여 처음부터 시작합니다. 시작 파일을 제공하지 않습니다.

C++ Windows 콘솔

빈 솔루션

프로젝트 없는 빈 솔루션 만들기

기타

원하는 항목을 찾지 못하셨나요?

추가 도구 및 기능 설치

뒤로(B) 다음(N)

새 프로젝트 구성

빈 솔루션 기타

솔루션 이름(N)

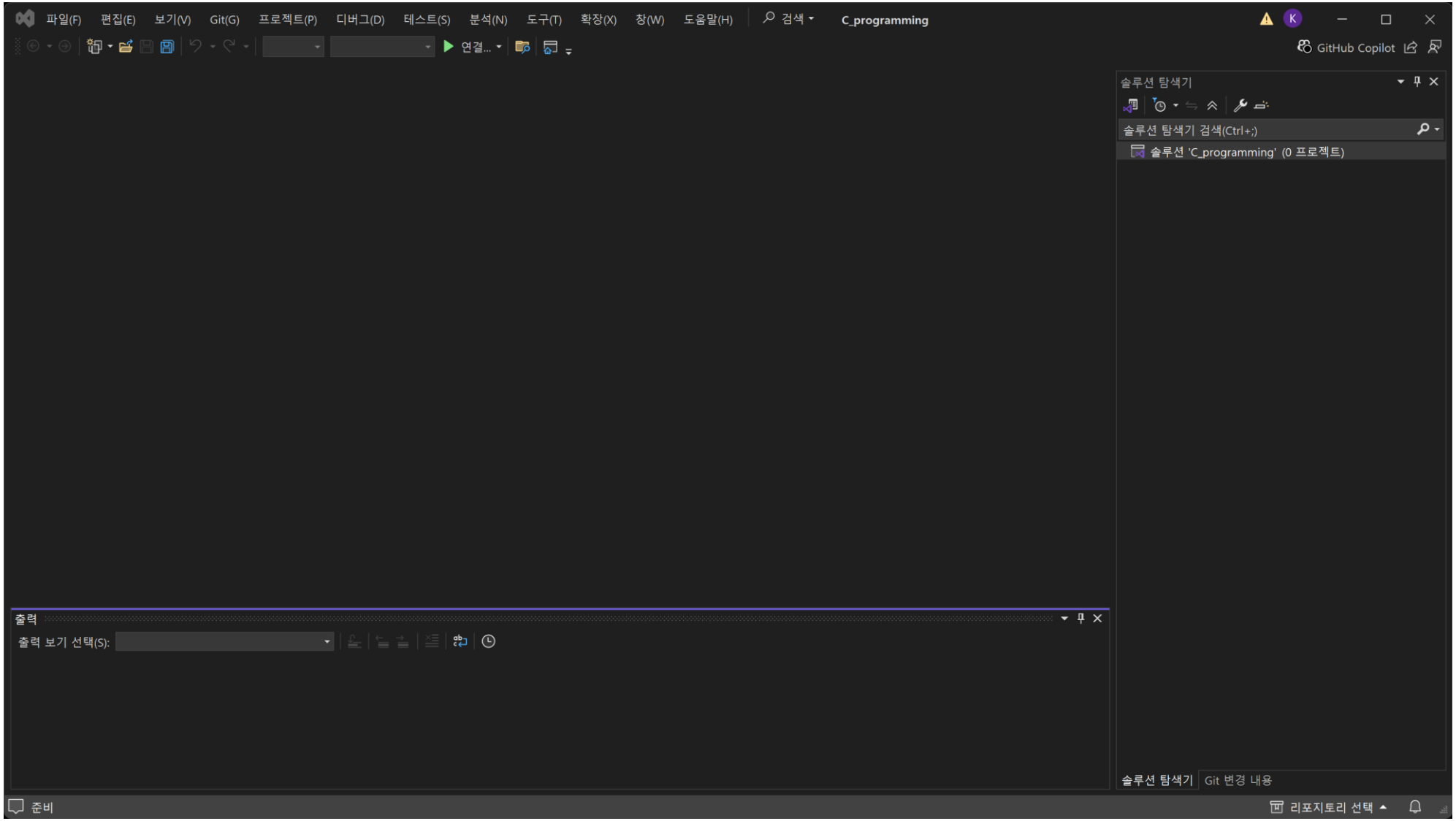
C_programming

위치(L)

C:₩Users₩kyungho₩source₩repos ...

"C:₩Users₩kyungho₩source₩repos₩C_programming₩"에 솔루션이(가) 만들어집니다.

뒤로(B) 만들기(C)



파일 > 새로만들기 > 프로젝트

or ctrl + shift + N

새 프로젝트 만들기

최근 프로젝트 템플릿(R)

빈 솔루션

빈 프로젝트 C++

템플릿 검색(Alt+S)(S)

모든 언어(L) 모든 플랫폼(P) 모든 프로젝트 형식(T)

Vue and ASP.NET Core 새로 만들기
프런트 엔드 Vue 프로젝트와 백 엔드 ASP.NET Core 프로젝트가 있는 전체 스택 애플리케이션
TypeScript Windows macOS Linux 웹

내보내기가 있는 동적 연결 라이브러리(DLL) 새로 만들기
실행 중인 여러 Windows 앱 간에 공유할 수 있는 .dll을 빌드하세요.

빈 프로젝트
Windows용 C++를 사용하여 처음부터 시작합니다. 시작 파일을 제공하지 않습니다.
C++ Windows 콘솔

빈 솔루션
프로젝트 없는 빈 솔루션 만들기
기타

원하는 항목을 찾지 못하셨나요?
추가 도구 및 기능 설치

다음(N)

새 프로젝트 구성

빈 프로젝트 C++ Windows 콘솔

프로젝트 이름(J)

lab01_hello_world

위치(L)

C:₩Users₩kyungho₩source₩repos₩C_programming ...

솔루션(S)

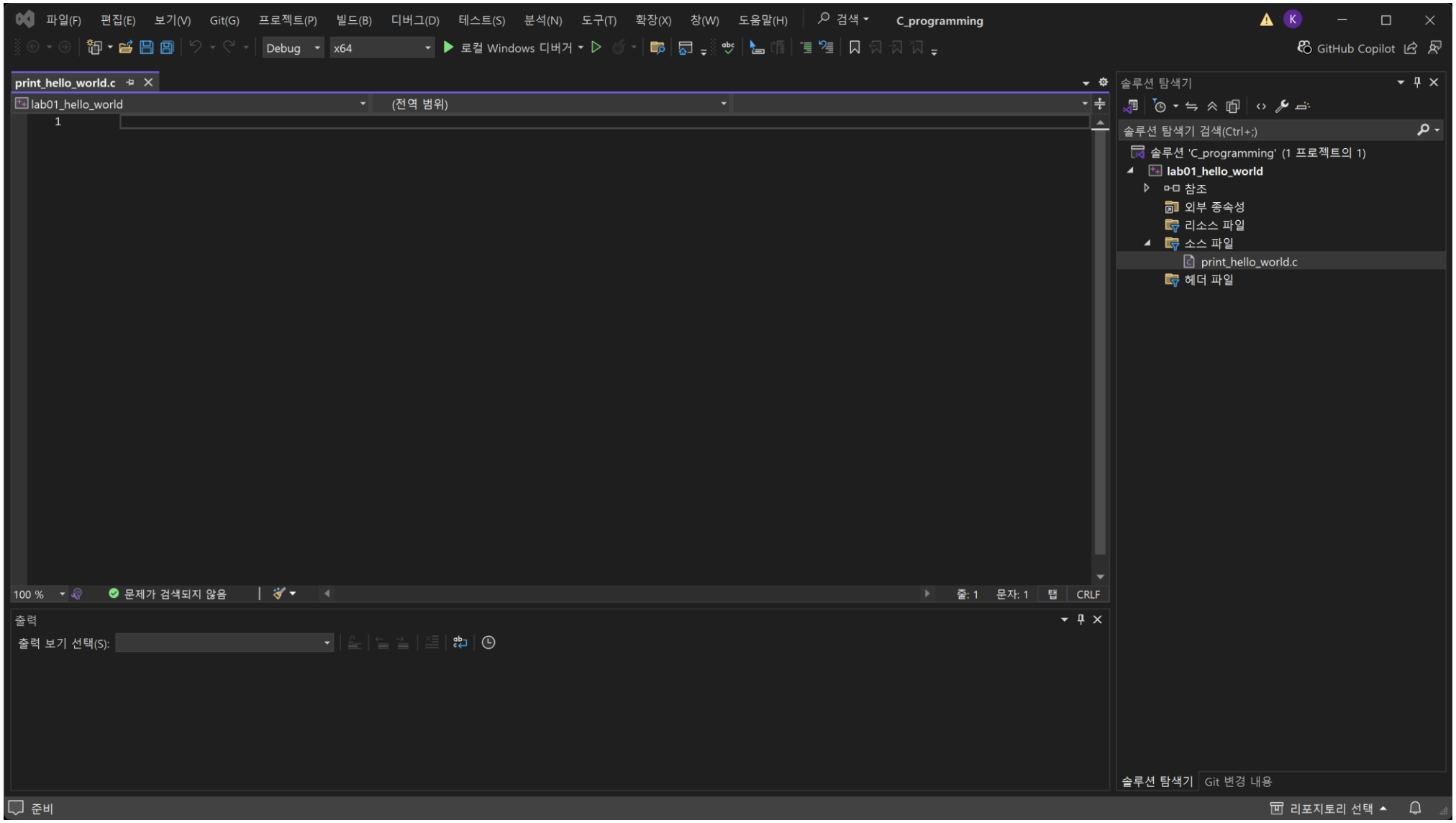
솔루션에 추가

솔루션 이름(M)

C_programming

"C:₩Users₩kyungho₩source₩repos₩C_programming₩lab01_hello_world₩"에 프로젝트이(가) 만들어집니다.

뒤로(B) 만들기(C)


파일(F) 편집(E) 보기(V) Git(G) 프로젝트(P) 빌드(B) 디버그(D) 테스트(S) 분석(N) 도구(T) 확장(X) 창(W) 도움말(H)
검색
C_programming
Debug
x64
로컬 Windows 디버거
GitHub Copilot
print_hello_world.c
lab01_hello_world
(전역 범위)
1
100 %
문제가 검색되지 않음
줄: 1
문자: 1
탭
CRLF
출력
출력 보기 선택(S):
솔루션 탐색기
솔루션 탐색기 검색(Ctrl+;)
솔루션 'C_programming' (1 프로젝트의 1)
lab01_hello_world
참조
외부 종속성
리소스 파일
소스 파일
print_hello_world.c
헤더 파일
솔루션 탐색기
Git 변경 내용
준비
리포지토리 선택

파일(F) 편집(E) 보기(V) Git(G) 프로젝트(P) 빌드(B) 디버그(D) 테스트(S) 분석(N) 도구(T) 확장(X) 창(W) 도움말(H) 검색 C_pr...mming

Debug x64 로컬 Windows 디버거 GitHub Copilot

print_hello_world.c

lab01_hello_world (전역 범위) main(void)

```c
#include<stdio.h>
int main(void)
{
    printf("Hello, world!\n");
    return 0;
}
```

100 % 문제가 검색되지 않음

출력

출력 보기 선택(S): 빌드

```
오후 12:40에 빌드를 시작함...
1>------ 빌드 시작: 프로젝트: lab01_hello_world, 구성: Debug x
1>print_hello_world.c
1>lab01_hello_world.vcxproj -> C:WUsersWkyunghoWsourceWreposWC_
========== 빌드: 1개 성공, 0개 실패, 0개 최신 상태, 0개 건너뜀
========== 빌드이(가) 오후 12:40에 완료되었으며, 00.999 초이(가
```

솔루션 탐색기

솔루션 탐색기 검색(Ctrl+;)

- 솔루션 'C_programming' (1 프로젝트의 1)
  - lab01_hello_world
    - 참조
    - 외부 종속성
    - 리소스 파일
    - 소스 파일
      - print_hello_world.c
    - 헤더 파일

Microsoft Visual Studio 디버그 콘솔

```
Hello, world!

C:\Users\kyungho\source\repos\C_programming\x64\Debug\lab01_hello_world.exe(프로세스 1008
)이(가) 0 코드(0x0)와 함께 종료되었습니다.
이 창을 닫으려면 아무 키나 누르세요...
```

솔루션 탐색기 Git 변경 내용

빌드 성공 리포지토리 선택

파일(F) 편집(E) 보기(V) Git(G) 프로젝트(P) 빌드(B) 디버그(D) 테스트(S) 분석(N) 도구(T) 확장(X) 창(W) 도움말(H) 검색 C_pr...mming

Debug x64 로컬 Windows 디버거 GitHub Copilot

practice_1.c print_hello_world.c

lab01_hello_world (전역 범위)

```c
Your name
Your student ID number
Your department
*/

#if 0
  Practice 1. print your info
  Write a program that prints:
  Your name
  Your student ID number
  Your department
#endif

#include <stdio.h>

int main(void)
{
    printf("Name:   Kim Minsu\n");
    printf("ID:     202012345\n");
    printf("Dept:   Embedded Systems Engineering\n");
    return 0;
}
```

100 % 문제가 검색되지 않음 줄: 28 문자: 13 열: 16 탭 CRLF

오류 목록

전체 솔루션 | 2 오류 | 0 경고 | 0 메시지 | 빌드 + IntelliSense | 검색 오류 목록

코드	설명	프로젝트	파일	줄	세부...
LNK2005	main이(가) practice_1.obj에 이미 정의되어 있습니다.	lab01_hello_world	print_hello_world.obj	1	
LNK1169	여러 번 정의된 기호가 있습니다.	lab01_hello_world	lab01_hello_world.exe	1	

오류 목록 출력

솔루션 탐색기

솔루션 탐색기 검색(Ctrl+;)

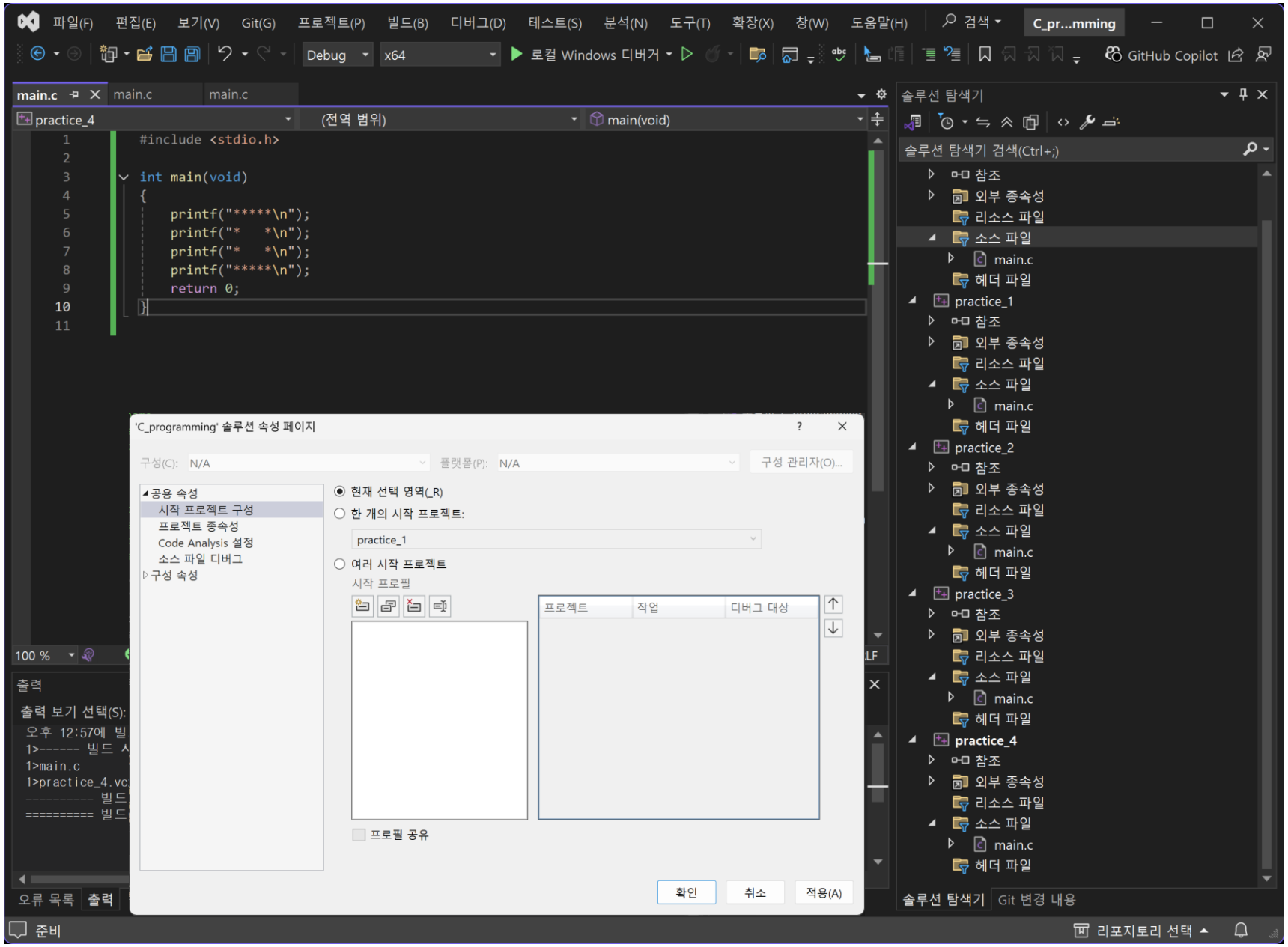
솔루션 'C_programming' (1 프로젝트의 1)
- lab01_hello_world
  - 참조
  - 외부 종속성
  - 리소스 파일
  - 소스 파일
    - practice_1.c
    - print_hello_world.c
  - 헤더 파일

솔루션 탐색기 Git 변경 내용

준비 리포지토리 선택

주의: 한 프로젝트에는 하나의 main 함수

한 솔루션에 여러 프로젝트 있을 때 빌드하고 싶은 프로젝트를 시작프로젝트로 설정



Hello, World! — A Programming Tradition

- The very first program almost every programmer writes
- Originated in 'The C Programming Language' (1978)
- Why this program?
  - Simplest possible complete program
  - Proves your environment works
  - Introduces the core structure of C

- Let's write it together!

Source Files

소스파일은 텍스트파일 형식이므로 모든 편집기로 읽거나 작성할 수 있으나 아래한글이나 워드와 같은 문서편집기에서는 반드시 텍스트 형태로 저장해야 한다.

```c
# include <stdio.h>

int main()
{
    printf("hello, world\n");

    return 0;
}
```

C 소스파일: *.c

소스파일은 모든 편집기로 읽을 수 있으나 아래한글이나 워드로 작성된 일반 문서는 내부 형식이 다르므로 작성한 편집기로만 읽을 수 있다.

아래한글로 작성된 문서는 .hwp 라는 확장자이듯, 소스파일은 프로그래밍 언어에 따라 고유한 확장자를 갖는데, C 언어는 .c이며 자바는 .java 그리고 C++는 .cpp 이다.

아래한글 파일: *.hwp

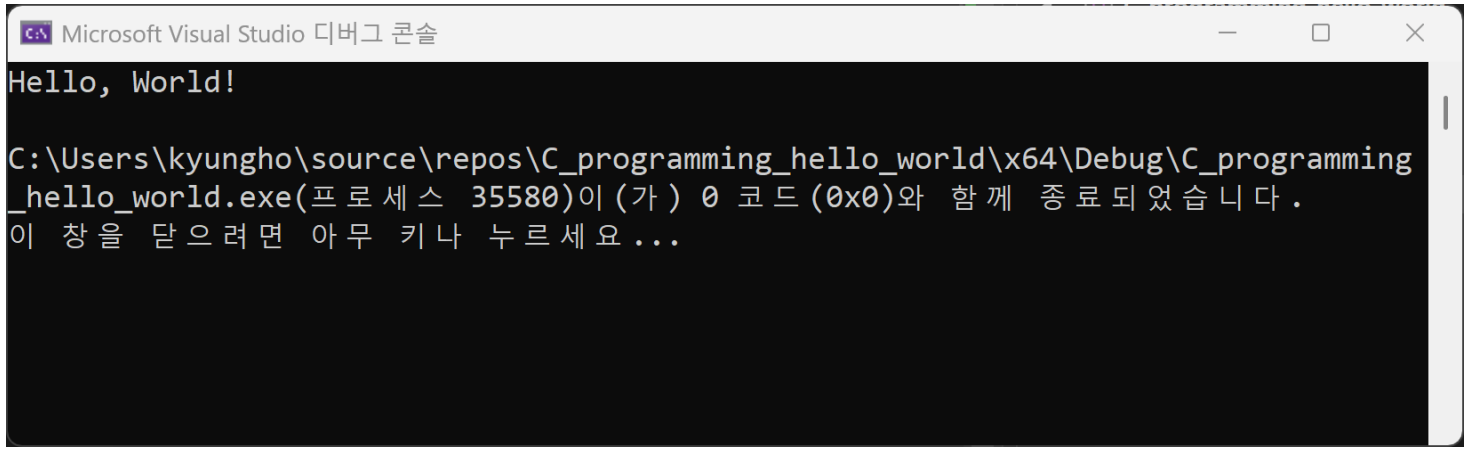
C source files have the .c extension — they are plain text files

Hello World — Complete Code

Only 7 lines — but a complete, working program!

```c
/* hello.c - My first C program */
#include <stdio.h>

int main(void)
{
    printf("Hello, World!\n");
    return 0;
}
```



Understanding: Comments

- /* hello.c - My first program */

- Comments are ignored by the compiler
- Purpose: explain code to human readers
- /* ... */ : multi-line comment (C89/C90)
- // ... : single-line comment (C99+)

- Good habit: comment every file with its purpose

Comment Examples

```c
/* This is a
   multi-line comment */

// This is a single-line comment

int x = 5;  /* inline comment */

/* BAD: commenting out code is confusing
int y = 10; /* some note */
int z = 20;
*/
```

```c
#if 0
int y = 10;
int z = 20;
#endif
```

Semicolons & Indentation

- Semicolons ;
  - Mark the end of every statement in C
  - Forgetting a semicolon is the #1 beginner mistake!
  - Be careful: colon : ≠ semicolon ;
- Indentation
  - Code inside { } should be indented (4 spaces or 1 tab)
  - The IDE auto-indents for you
  - Improves readability — essential, not optional!

Understanding: #include

- #include <stdio.h>

- Preprocessor directive (runs before compilation):
- stdio.h = Standard Input / Output header
- Provides: printf(), scanf(), and more
- < > means: system header (installed with compiler)

- Without this line, printf() is not available!

Common Header Files

- #include <stdio.h> // printf, scanf, file I/O
- #include <stdlib.h> // malloc, free, exit
- #include <string.h> // strlen, strcpy, strcmp
- #include <math.h> // sqrt, pow, sin, cos
- #include <time.h> // time, clock

- Each header provides a group of related functions

Understanding: main()

```
int main(void)
{
    /* your code goes here */
   return 0;
}
```

```
int main(int argc, char *argv[])
{
    /* your code goes here */
   // argc = number of inputs
   // argv = input character string
   return 0;
}
```

Every C program must have exactly one main():

- Execution always starts from main()
- int = the function returns an integer
- void = takes no arguments (for now)
- { } = function body

Understanding: printf()

- printf("Hello, World!₩n");

- Prints formatted text to the screen:
- "..." = string literal (text between quotes)
- ₩n = newline character (moves to next line)
- ; = statement terminator (required!)
- Defined in <stdio.h>

Escape Sequences in Strings

Special characters inside string literals

```
printf("New line:\n");          // moves to next line
printf("Tab:\there");           // horizontal tab
printf("Quote: \"hi\"");        // double quote
printf("Backslash: \\");        // backslash
printf("Beep!\a");              // terminal beep
```

puts() — Simple String Output

- puts() is a simpler alternative to printf() for plain text
  - Prints the string and automatically adds a newline
  - No format specifiers — just plain strings
  - Also defined in <stdio.h>
- Comparison:
  - printf("Hello₩n"); // must add ₩n yourself
  - puts("Hello"); // ₩n added automatically
- Use printf() for formatted output (%d, %f, %c)
- Use puts() for simple string output

Understanding: return 0

- return 0;
- Exit status returned to the operating system:
  - 0 = program finished successfully
  - Non-zero = program encountered an error
- Required at the end of main() in C89/C90
- The OS can use this value to check for errors

General C Program Structure

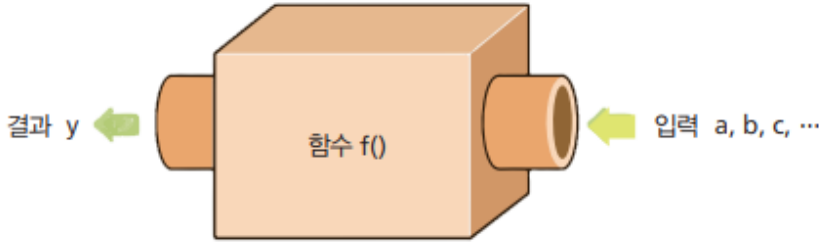
- /* File header comment */
- #include <...> ← header files
- #define CONSTANT ... ← constants (optional)

- int main(void)
- {
- /* statements */
- return 0;
- }

- /* other functions (optional) */

Functions in C — A First Look

- A C program is made up of functions
- Function = a named block of code that performs a task
  - Input (arguments) → [Function body] → Output (return value)
- Two kinds of functions:
  - User-defined functions — you write them yourself
  - Library functions — pre-built by the system (printf, scanf)
- main() is a special function:
  - Starting point of every C program — like a car's ignition key
  - We will study functions in depth in Week 7!



$$y = f(a, b, c, \cdots)$$

Variables

- Variable = named storage location in memory

- Rules:
- Must be declared before use
- Syntax: type name;
- Name: letters, digits, underscore (not starting with digit)
- C is case-sensitive: age ≠ Age ≠ AGE

Basic Data Types

- int → whole numbers (42, -10, 0)
- float → decimal numbers (3.14, -0.5)
- double → higher precision (3.14159265...)
- char → single character ('A', 'z', '5')

- We will study all data types in depth in Week 2!

Declaring and Using Variables

%d = int, %f = float, %c = char

```c
#include <stdio.h>

int main(void)
{
    int students = 30;
    float average = 85.5;
    char grade = 'A';

    printf("Students: %d\n", students);
    printf("Average:  %.1f\n", average);
    printf("Grade:    %c\n", grade);
    return 0;
}
```

Understanding Errors & Debugging

What is Debugging?

- Debugging = finding and fixing errors in code

- Every programmer encounters bugs — it is normal!
- Even professional developers spend ~50% debugging
- Learning to debug efficiently is crucial

- Good news: your IDE helps you find bugs!

Types of Errors

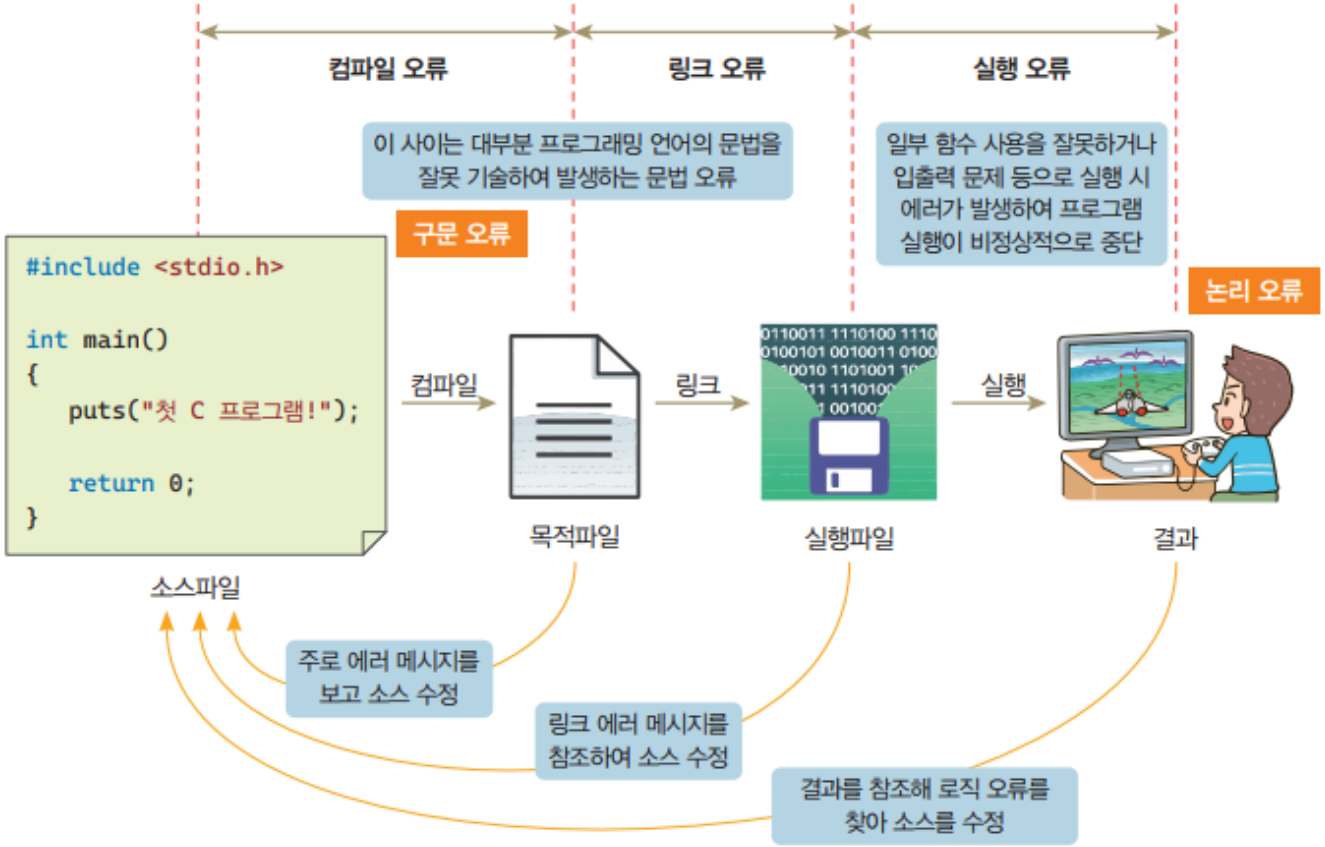


그림 2-5 디버깅의 순환 과정

Compile errors → Link errors → Runtime errors → Logic errors

Three Types of Errors

1. Syntax (Compile) Error

- Code breaks grammar rules → won't compile
- IDE underlines the problem immediately

2. Runtime Error

- Program compiles but crashes while running
- Example: dividing by zero

3. Logic Error

- Program runs but gives wrong answer
- Hardest to find — you must test carefully

Understanding Link Errors

- Link errors: the linker cannot find a function or symbol
- Most common cause: typos in function names!
- Example:
  - print("Hello"); ← WRONG: 'print' does not exist
  - printf("Hello"); ← CORRECT: 'printf' is the real function
- The compiler won't catch this — it only checks syntax
- The linker fails because 'print()' is not in any library
- Tip: 'unresolved external symbol' or 'undefined reference'
  - → check your function names carefully!

Common Syntax Errors

Read error messages carefully — they show the line number!

```
// Missing semicolon
printf("Hello")   // ERROR: missing ;

// Typo in function name
Printf("Hello");   // ERROR: it is printf (lowercase)

// Wrong quotes for strings
printf('Hello');   // ERROR: use double quotes ""

// Missing closing brace
int main(void) {
    return 0;
// ERROR: missing }
```

Reading IDE Error Messages

- When you build, the IDE shows errors in the Output panel:

```
hello.c(5): error: expected ';' before '}'
  ↑ file    ↑ line  ↑ what is wrong
```

Tips:

- Fix the FIRST error first — others may disappear
- Click the error message to jump to that line
- Google the error message if unclear

Using the IDE Debugger

- Visual Studio / VS Code both have built-in debuggers:

Key features:

- Breakpoint: pause execution at a specific line
- Step Over (F10): run one line at a time
- Step Into (F11): go inside a function call
- Watch: monitor variable values as they change

- We will practice this in the lab!

Debugging Tips for Beginners

- Always build after every small change
- Fix the first error first, then rebuild
- Read error messages — don't just guess
- Use printf() to print variable values

```
printf("x = %d₩n", x);
```

- Test with simple, known inputs first
- Ask for help early — don't struggle alone for hours

C for Embedded Systems

What are Embedded Systems?

- Computers built inside devices for a specific purpose:

  Not general-purpose like a PC

  Dedicated, often real-time operation

  Usually interact directly with hardware

- You use them every day without knowing!

Embedded Systems Examples

- Automotive: ABS brakes, airbags, engine control
- Home: Washing machine, microwave, TV remote
- Medical: Pacemakers, insulin pumps, MRI machines
- Industrial: Robots, conveyor systems, sensors
- Consumer: Smartphones, cameras, wireless earbuds

- Billions of embedded devices are running right now!

Why Use C for Embedded Systems?

- Efficiency:
- Small code size (microcontrollers have KB of flash)
- Fast execution (no interpreter overhead)
- Hardware access:
- Read/write hardware registers directly
- Bit-level manipulation
- Portability:
- Available on almost every microcontroller

Embedded vs Desktop Programming

- Embedded constraints:
- RAM: kilobytes (not gigabytes!)
- Flash: kilobytes to megabytes
- No operating system (often)
- Real-time deadlines
- No keyboard/screen (usually)

- C gives you the control to work within these limits

Embedded Development (Preview)

- In later courses you will use:

- Cross-compiler: compile on PC, run on microcontroller
- arm-none-eabi-gcc (ARM Cortex-M chips)
- avr-gcc (AVR / Arduino chips)

- The C skills from this course transfer directly!
- Same language, slightly different tools

Practice Problems

Practice 1: Print Your Info

- Write a program that prints:
- Your name
- Your student ID number
- Your department

- Requirements:
- Use a separate printf() for each line
- Each line must end with ₩n

Practice 2: Simple Arithmetic

- Declare two integer variables a = 15, b = 27
- Calculate and print:

```
Sum:        a + b

Difference: a - b

Product:    a * b
```

- Each result on its own line

Practice 3: Float Formatting

- Declare: float pi = 3.141593;

- Print pi three times with different precision:

```
Default (6 decimal places): 3.141593

2 decimal places:           3.14

4 decimal places:           3.1416
```

- Hint: use %f, %.2f, %.4f

Practice 4: Draw a Pattern

- Print the following shape using printf():

```
*****

*  *

*  *

*****
```

- Use 4 printf() statements — one per line
- Pay attention to spaces inside the quotes!

Practice 5: Fix the Bug

- The following code has 3 errors. Find and fix them:

```
#include <stdio.h>

int main(void)
{
    Printf("Hello World\n")
    return 0
}
```

Practice 6: Personal Introduction

- Write a program that prints a short self-introduction:

- Expected output (use your own info):

```
"Hello! My name is [name]."
"I am [age] years old."
"I study Embedded Systems Engineering."
"I want to become an embedded software engineer!"
```

- Bonus: add a blank line between sections

Practice 7: Escape Sequences

- Print the following output using printf():

```
Name      Score

Kim       95



"End of list"
```

- Requirements:
- Use ₩t for tab alignment between columns
- Use ₩" for literal double-quote characters
- Use 4 separate printf() statements

Practice 8: Using puts()

- Rewrite the following code using puts() instead of printf():

```
printf("C Programming\n");
printf("Week 1 Practice\n");
printf("Hello, World!\n");
```

Hint: puts() adds ₩n automatically

Practice 9: char Data Type

- Declare: char grade = 'A';

- Print the following output:

```
Grade (character): A

ASCII value:       65
```

- Hint:
- %c prints the variable as a character
- %d prints the variable as an integer (ASCII code)

Practice 10: All Four Basic Types

- Declare variables of all 4 basic data types:

```
int    score   = 98
float  gpa     = 3.75f
double temp    = 36.5837
char   initial = 'K'
```

- Print each with the correct format specifier:

```
Score:    98
GPA:      3.75      (use %.2f)
Temp:     36.58     (use %.2f)
Initial:  K
```

Practice 11: Fix the Bug #2

- This code has 3 bugs. Find and fix them all:

```
#include <stdio.h>

int main(void)
{
    int     score    = 95

    printf("Score %f\n", score);
    Printf(”Passed!\n", score);
    return 0;
}
```

Practice 12: Diamond Pattern

- Print this diamond pattern using exactly 5 printf() statements:

```
  *
 ***
*****
 ***
  *
```

- One printf() per line
- Pay careful attention to spaces before *

Common Beginner Mistakes

- ❌ Missing semicolons at end of statements
- ❌ Forgetting #include <stdio.h>
- ❌ Using single quotes for strings ('Hello')
- ❌ Using Printf instead of printf
- ❌ Uninitialized variables (random garbage value)
- ❌ Ignoring compiler error messages

✅ Fix one error at a time — rebuild after each fix

This Week's Lab

Step 1: Install your IDE

- Windows: Visual Studio Community
- Mac: VS Code + C/C++ + Code Runner extensions

Step 2: Write Hello World and run it

Step 3: Solve Practice Problems 1–6

Step 4: Experiment — try breaking the code on purpose

- → Read error messages and fix them

How to Succeed in C Programming

- Type the code yourself — never just copy-paste
- Practice every day, even 30 minutes is enough
- Make mistakes intentionally and fix them
- Read error messages before asking for help
- Use the IDE debugger, not just guessing
- Help classmates — explaining reinforces learning

Week 1 Summary

- C: created 1972, still essential for systems programming
- IDE: Visual Studio (Win) or VS Code + extensions (Mac)
- Compilation: source code → compiler → executable
- Every C program: #include → main() → statements → return 0
- printf() prints to screen; ₩n starts a new line
- Errors: syntax (compile) · runtime · logic

What You Should Know Now

✓ What C is and why it matters for embedded systems

✓ How to install and use your IDE

✓ The structure of a C program

✓ How to use printf() and format specifiers

✓ The three types of errors

✓ How to read IDE error messages

Next Week: Variables & Data Types

- Deep dive into data types:
- int, float, double, char
- signed vs unsigned
- Size and range of each type
- Type conversion (implicit and explicit)
- Constants: const and #define

- Come to the lab with your IDE installed and working!

Questions?

- Thank you for your attention!

- Office hours: anytime through email
- Email: kyungho.won@inu.ac.kr
- LMS: course materials and assignment submissions

- See you in the lab — bring your laptop!

Online Judge

임베디드시스템공학과 온라인 저지 게시판 문제 모음 채점 상황 열람 순위 대회 로그인

등록 정보

사용자 ID:	HJO202301676	이니셜 + 학번
닉네임:	황주옥	성명
비밀번호:	••••••••••••	비밀번호
비밀번호 반복 입력:	••••••••••••	
소속:		
이메일:	ghkdwndhr227@inu.ac.kr	이메일

Submit

Reset

임베디드시스템공학과 온라인 저지 | 게시판 | 문제 모음 | 채점 상황 열람 | 순위 | 대회 | HJO202301676

1 2 3 4 5 6 7 8 9 10

Problem ID | Go | Keywords Title or Source | Search

문제 ID	문제명	출처	AC	제출
1000	첫번째 문제		715	1104
1001	과일옮기기		200	283
1002	Hello World 두 줄에 출력해 보기		439	677
1003	따옴표 안에 Hello 출력하기		374	457
1004	큰 따옴표 안에 Hello World 출력하기		347	690
1005	채점서버 테스트 문제		223	300
1006	[문법-출력] 출력하기01	정보컴퓨터교사	426	490
1007	[문법-출력] 출력하기02	정보컴퓨터교사	395	449
1008	[문법-출력] 출력하기03	정보컴퓨터교사	405	538
1009	[문법-출력] 출력하기04	정보컴퓨터교사	383	467
1010	[문법-출력] 출력하기05	정보컴퓨터교사	368	520
1011	[문법-출력] 출력하기06	정보컴퓨터교사	373	705
1012	[문법-출력] 출력하기08	정보컴퓨터교사	92	521
1013	[문법-출력] 출력하기07	정보컴퓨터교사	336	515
1014	[문법-입출력] 정수 한 개 입력받아 그대로 출력하기	정보컴퓨터교사	268	327
1015	[문법-입출력] 문자 한 개 입력받아 그대로 출력하기	정보컴퓨터교사	226	265
1016	[문법-출력] 출력하기09	정보컴퓨터교사	107	445
1017	[문법-입출력] 실수 한 개 입력받아 그대로 출력하기	정보컴퓨터교사	200	607

임베디드시스템공학과 온라인 저지 게시판 문제 모음 채점 상황 열람 순위 대회 HJO202301676

1002: Hello World 두 줄에 출력해 보기

시간 제한: 1 Sec 메모리 제한: 128 MB
제출: 677 해결 문제 수: 439
[제출][채점 상황 열람][게시판]

문제 설명

이번에는 줄을 바꿔 문장을 출력하는 출력을 연습해보자.
다음과 같이 줄을 바꿔 출력 해야한다.

Hello
World

두 줄에 걸쳐서 줄을 바꿔 출력

입력

출력

Hello
World

출력 예시

```
Hello
World
```

도움말

출처

[제출][채점 상황 열람]

임베디드시스템공학과 온라인 저지 게시판 문제 모음 채점 상황 열람 순위 대회 HJO202301676

Problem **1002**

Language: C

```
#include <stdio.h>

int main() {

printf("Hello₩nWorld");

return 0;
}
```

Position: Ln 8, Ch 2 Total: Ln 8, Ch 64

Togg

출력:

```
Hello
World
```

입력:

출력:

```
SHOULD BE:
Hello
World
```

제출 TEST 状态 重置

임베디드시스템공학과 온라인 저지 게시판 문제 모음 채점 상황 열람 순위 대회 HJO202301676

Problem **1002**

Language: C

12 pt CPP

```c
#include <stdio.h>

int main() {
printf("Hello");

return 0;
}
```

Position: Ln 7, Ch 2 Total: Ln 7, Ch 56

☑ Toggle editor

입력:

출력:

```
SHOULD BE:
Hello
World
```

제출 TEST 状态 重置